

```
import torch
import pandas as pd
import re
from transformers import AutoTokenizer, AutoModelForCausalLM
from google.colab import files
```

```
uploaded=files.upload()
```

 train-00000...001.parquet
train-00000-of-00001.parquet(n/a) - 2677259 bytes, last modified: 2/23/2026 - 100% done
 Saving train-00000-of-00001.parquet to train-00000-of-00001.parquet

```
file_name=list(uploaded.keys())[0]
data=pd.read_parquet("train-00000-of-00001.parquet")
print("Total Samples:",len(data))
print(data.head())
```

Total Samples: 7473

```
question \
0 Natalia sold clips to 48 of her friends in Apr...
1 Weng earns $12 an hour for babysitting. Yester...
2 Betty is saving money for a new wallet which c...
3 Julie is reading a 120-page book. Yesterday, s...
4 James writes a 3-page letter to 2 different fr...
```

```
answer
0 How many clips did Natalia sell in May? ** Nat...
1 How much does Weng earn per minute? ** Weng ea...
2 How much money does Betty have in the beginnin...
3 How many pages did Maila read today? ** Maila ...
4 How many pages does he write each week? ** He ...
```

```
device="cuda" if torch.cuda.is_available() else "cpu"
print("Using device:",device)
```

Using device: cuda

```
model_name="TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tokenizer=AutoTokenizer.from_pretrained(model_name)
model=AutoModelForCausalLM.from_pretrained(model_name, dtype=torch.float16 if device=="cuda" else torch.float32,
```

low_cpu_mem_usage=True).to(device)

model.eval()

Loading weights: 100%

201/201 [00:03<00:00, 55.30it/s, Materializing param=model.norm.weight]

```

LlamaForCausalLM(
  (model): LlamaModel(
    (embed_tokens): Embedding(32000, 2048)
    (layers): ModuleList(
      (0-21): 22 x LlamaDecoderLayer(
        (self_attn): LlamaAttention(
          (q_proj): Linear(in_features=2048, out_features=2048, bias=False)
          (k_proj): Linear(in_features=2048, out_features=256, bias=False)
          (v_proj): Linear(in_features=2048, out_features=256, bias=False)
          (o_proj): Linear(in_features=2048, out_features=2048, bias=False)
        )
        (mlp): LlamaMLP(
          (gate_proj): Linear(in_features=2048, out_features=5632, bias=False)
          (up_proj): Linear(in_features=2048, out_features=5632, bias=False)
          (down_proj): Linear(in_features=5632, out_features=2048, bias=False)
          (act_fn): SiLUActivation()
        )
        (input_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
        (post_attention_layernorm): LlamaRMSNorm((2048,), eps=1e-05)
      )
    )
    (norm): LlamaRMSNorm((2048,), eps=1e-05)
    (rotary_emb): LlamaRotaryEmbedding()
  )
  (lm_head): Linear(in_features=2048, out_features=32000, bias=False)
)

```

```

def build_prompt(question,mode="zero_shot",exemplars=None):
    if mode=="zero_shot":
        return(
            "Solve the folowing math problem.\n"
            "Give only the final numeric answer.\n\n"
            f"Question: {question}\n"
            "Let's think step by step."
        )
    else:
        raise ValueError("Invalid prompting mode")

```

```

def generate_answer(prompt,max_new_tokens=128):
    inputs=tokenizer(prompt,return_tensors='pt').to(device)

```

```

with torch.no_grad():
    outputs=model.generate(
        **inputs,
        max_new_tokens=max_new_tokens,
        do_sample=False)
return tokenizer.decode(outputs[0],skip_special_tokens=True)

```

```

def extract_last_number(text):
    numbers=re.findall(r"-?\d+\.\d*",text)
    return numbers[-1] if numbers else None

```

```

def get_few_shot_examples(k=3):
    examples=[]
    for i in range(k):
        q=data.iloc[i]["question"]
        a=data.iloc[i]["answer"].split("###")[-1].strip()
        examples.append((q,a))
    return examples
few_shot_examples=get_few_shot_examples(k=3)

```

```

def evaluate_mode(mode,n_samples=50):
    correct=0
    hallucinations=0
    for i in range(n_samples):
        question=data.iloc[i]["question"]
        gt_answer=data.iloc[i]["answer"].split("###")[-1].strip()
        if mode=="few_shot":
            prompt=build_prompt(question,mode,few_shot_examples)
        else:
            prompt=build_prompt(question,mode)
        output=generate_answer(prompt)
        pred=extract_last_number(output)
        gtt=extract_last_number(gt_answer)
        if pred is None:
            hallucinations+=1
        elif pred==gtt:
            correct+=1
        if i%10==0:
            print(f"[{mode}] Processed {i+1}/{n_samples}")
    accuracy=correct/n_samples
    hallucination_rate=hallucinations/n_samples
    print("\n-----")
    print(f"Mode: {mode}")

```

```
print(f"Mode: {mode}")
print(f"Accuracy:{accuracy:.3f}")
print(f"Hallucination Rate:{hallucination_rate:.3f}")
print("-----\n")
return accuracy,hallucination_rate
acc_zero,hall_zero=evaluate_mode("zero_shot")
# acc_few,hall_few=evaluate_mode("few_shot") # Commented out until 'cot' mode is handled or removed
# acc_cot,hall_cot=evaluate_mode("cot") # Commented out until 'cot' mode is handled or removed
```

```
[zero_shot] Processed 1/50
[zero_shot] Processed 11/50
[zero_shot] Processed 21/50
[zero_shot] Processed 31/50
[zero_shot] Processed 41/50
```

```
-----
Mode: zero_shot
Accuracy:0.040
Hallucination Rate:0.020
-----
```